AGENT-READY PRODUCT SPECIFICATION

Road Report AI

Version: 2.0 (Draft)
Build Plan: 5 Phases
Stack: React 19, FastAPI, PostgreSQL, PyTorch
Auth: OAuth
Read this entire specification before writing a single line of code. This document is the single source of truth for all AI coding agents.

Table of Contents

1. Product Overview

Road Report AI is a web application that uses machine learning and live roadway context to estimate crash risk for real Texas roads and corridors. The platform accepts a road or location query, geocodes it into coordinates, sends those coordinates to the backend prediction service, and returns a structured safety report with a risk score, tier, and summary.

1.1 Target Users

- **First responders:** Those that need to be able to prioritize higher risk areas to respond to.
- **City planners:** Those that are able to improve the quality of roads to reduce risk.
- **General citizens (later phase):** Commuters that might want to be aware of risky roads.

1.2 Core Features

- Geocoded road and location search.
- Backend-connected risk report generation using live API requests.
- Interactive map-based visualization for searched corridors.
- Safety report page with risk score, tier, summary, and incident-oriented presentation.
- Methodology page describing the predictive framework and model logic.
- Documentation page covering data sources, formula, stack, and team ownership.
- Dark mode support and browser history/back-button support for in-app page navigation.

2. Tech Stack

All technology decisions below are fixed for the current phase.

Layer	Technology	Notes / Constraints
Frontend	React 19, TypeScript, Vite, Tailwind CSS 4, MapLibre GL, motion, lucide-react, recharts	Sovereign-observer folder is the primary frontend.
Mapping	MapLibre + OSM/Overpass + Nominatim	
Backend	FastAPI, PostgreSQL, PyTorch, Pandas + NumPy	REST API
AI/ML	Pytorch + Pandas + NumPy	Logistic Regression model currently integrated; PyTorch reserved for future model versions.
External APIs	US Weather.gov + TxDOT CRIS Query Tool + OpenStreetMap stack + Open-Meteo	Google Maps optional/future.
Auth & Validation	OAuth, Pydantic v2	

3. Project File Structure

```
Road-Report-AI-Backend/
├─ .cursorrules
├─ .env.example
├─ .gitignore
├─ README.md
├─ main.py
├─ requirements.txt
├─ app/
|   ├─ __init__.py
|   ├─ config.py
|   ├─ database.py
|   ├─ main.py
|   ├─ api/
|   |   ├─ __init__.py
|   |   ├─ deps.py
|   |   └─ routes/
|   |       ├─ __init__.py
|   |       ├─ health.py
|   |       ├─ reports.py
|   |       ├─ risk.py
|   |       └─ weather.py
|   ├─ models/
|   |   ├─ __init__.py
|   |   ├─ prediction.py
|   |   └─ user_report.py
|   ├─ schemas/
|   |   ├─ __init__.py
|   |   ├─ health.py
|   |   ├─ report.py
|   |   └─ risk.py
|   └─ services/
|       ├─ __init__.py
|       ├─ feature_columns.pkl
|       ├─ logistic_regression_model.pth
|       ├─ risk.py
|       └─ weather.py
├─ csv/
```

```
|   ├─ CreateTrainingData.py
|   ├─ csv_editing.py
|   ├─ csv_lookup.py
|   ├─ road_geocaching.py
|   ├─ README.md
|   └─ weather/
|      ├─ __init__.py
|      ├─ open_download.py
|      └─ open_preprocessing.py
└─ data_scraping/
   ├─ load_json.py
   └─ weather.py

Road-Report-AI-Frontend/sovereign-observer/
├─ .env.example
├─ .gitignore
├─ README.md
├─ index.html
├─ metadata.json
├─ package-lock.json
├─ package.json
├─ tsconfig.json
├─ vite.config.ts
└─ src/
   ├─ App.tsx
   ├─ constants.ts
   ├─ index.css
   ├─ main.tsx
   ├─ types.ts
   ├─ vite-env.d.ts
   └─ lib/
      ├─ risk.ts
      └─ utils.ts
```

4. Data Models & API Contracts

4.1 Model Training Data

```
model Data {
  City String;
  County String;
  CrashDate Date;
  CrashMonth int;
  CrashTime String;
  CrashYear int,
  DayOfWeek String;
  HourOfDay int,
  RoadClass String;
  RuralUrbanType String,
  Section int,
  StreetName String;
  SurfaceCondition String;
  WeatherCondition String;
  Crash Binary;
}
```

4.2 Weather Training Data

```
model weather{
  Date String,
  Hour int,
  Temperature_2m (°C) float,
  precipitation (mm) float,
  Weather_code (wmo code)int,
  Ice_flag Binary,
  Weather String,
  Road String;
}
```

4.3 Risk Predict Request

- **Required:** latitude (float), longitude (float)
- **Planned optional fields:** road_name, segment, road_class, weather_condition,

query_time_iso

4.4 Risk Predict Response

- **Implemented:** risk_score (0-1), latitude, longitude, message, total (0-100), tier, components, summary, advice, weather, warnings
- **Planned for later versions:** confidence/metadata fields as needed

5. Phase 1: MVP Setup & Data Processing (Weeks 1-2)

Requirement	Acceptance Criteria
A way to visualize the .csv data	A Python file with reusable functions to compare columns and rows. Any request by a user to compare data should result in a new function being added to this file.
Preprocessing weather training data	Transform the given NOAA data into usable Weather Training Data as formatted in 4.2. If weather and road conditions are not specified, use WMO codes to translate temperature and precipitation to road and weather conditions.
Add negative sampling to .csv file	Create a list of roads from the TxDOT crash records and pair them with hourly weather as generated in the previous step. This should combine all static columns from the crash records with the hourly data from weather. Once plausible negative samples are created, merge them with the positive samples.
Frontend & Backend Boilerplate	Produce the necessary files to integrate and pull from all needed resources using the tech stack. No additional tech without documentation.

6. Phase 2: AI Model Prototype & Frontend UI (Weeks 3-4)

Requirement	Acceptance Criteria
Splice data to prepare training	Randomly sample training data and load it into the model in batches. Some of the data from the training data file should be randomly kept as comparison data.
Create a basic logical regression AI model	A basic linear regression model should be created that takes in the values from Model Training data. This should be able to accept a road name and be able to output a general risk score (0-1) that predicts roughly how likely a crash is to occur.
Convert the Figma design to web application	Create a multi-page web application using React/Vite. These should be directly translated from a given figma document.
Attach basic map	Incorporate a basic UI of the map into the web page.

7. Phase 3: AI Output Refinement & Integrations (Weeks 5-6)

Requirement	Acceptance Criteria
Connect AI Model in backend to frontend	Create a way of querying the AI from the frontend application through basic API endpoints.
Fully integrate road API	Create functions to get data of the road (location, name, type) from the frontend. Modify the API created in the last task to accept the inputs from the map.
Integrate weather into training data pipeline	Create a function inside the csv directory that fetches weather from specific cities mentioned inside of the TxDOT crash data. This function should be able to be run automatically and be able to fetch all the cities mentioned in the TxDOT data or all the counties mentioned in the data.

8. Phase 4: Scaling & Real-time Context (Weeks 7-8)

Requirement	Acceptance Criteria
Create API to fetch current weather	Query Open-Meteo's API based on coordinates and convert WMO codes to NOAA definitions.
Add caching for Weather API	Store data server-side with a lifetime of 10 minutes. Add a locking mechanism to prevent stampeding.
Add bulk request to Weather API	Implement bulk requests to Open-Meteo. How long the api waits to send all the requests and how many requests can be sent at once should be controlled by a single variable.

9. Phase 5: Final Training Data Pipeline work (Weeks 9-10)

Requirement	Acceptance Criteria
Take the new Open-Meteo uploaded weather files and preprocess	Create a file with functions that loop through the directory weather_csvs and processes all the csv files inside to match the Weather Training Data format
Integrate the processed Weather Data with negative sampling generation	Convert the negative sample generation tool such that when weather is being generated for the negative sample the corresponding processed weather file is accessed and the correct weather for the sample's time is pulled.

10. API Reference

Base URL: /api/v1

- GET /health - Returns service health.
- POST /risk/predict - Generates risk prediction based on coordinates.
- POST /reports - Receives basic user reports.
- GET /health/model - Planned. Returns model metadata.
- GET /risk/model-metrics - Planned. Returns model performance metrics.
- GET weather/forecast - Fetches and caches current weather conditions and approximated road conditions based on Open-Meteos WMO codes

11. Error Handling Standards

```
{
  "error": {
    "Code": "VALIDATION_ERROR",
    "Message": "Request validation failed.",
    "details": []
  }
}
```

Codes: VALIDATION_ERROR, UNAUTHORIZED, NOT_FOUND, RATE_LIMITED, INTERNAL_ERROR

12. Non-Functional Requirements

- **Security:** Secure storage, restricted access (OAuth). Async DB and API calls.
- **Performance:** Model accuracy >= 75%. Risk score generated in < 2 seconds. No blocking operations in routes.
- **Code Quality:** Follow .cursorrules and skills.sh.
- **Scalability**: Ensure that when designing API, the design is around many people concurrently using the call at any given time.

13. Environment Variables

```
# Backend (.env)
DATABASE_URL=postgresql://postgres.trmfknldyvrjgmiibtqa:Itsnoonslayer123@aws-1-us-east-2.pooler.supabase.com:5432/postgres?ssl=require
GOOGLE_MAPS_API_KEY=
API_V1_PREFIX=/api/v1
DEBUG=
CORS_ORIGINS_CSV=https://road-report-ai-frontend.vercel.app,https://road-report-ai-frontend-41y6fnhxp-cchaiban21-6325s-projects.vercel.app
WEATHER_API_BASE_URL=https://api.weather.gov
WEATHER_USER_AGENT=roadreportai.prod,(insert email)
WEATHER_TIMEOUT_SECONDS=6.0
MODEL_FILE_PATH=app/services/logistic_regression_model.pth
MODEL_VERSION=log-reg-v1
FEATURE_COLUMNS_PATHS=app/services/feature_columns.pkl

# Frontend (.env)
VITE_API_BASE_URL=https://road-report-ai-backend.onrender.com/
```

14. Glossary

Term	Definition
Crash risk score	Numerical likelihood of an accident at a location.
TxDOT CRIS	Texas Department of Transportation Crash Records Information System.

15. Agent Instructions & Constraints

- **Phase Gates:** Complete each feature fully before moving to the next feature in the same phase.
- **Security Rules:** Never hardcode secrets, user IDs, or URLs. Never expose stack traces to users.
- **Decision Rules:** Follow .cursorrules. When the spec is ambiguous, implement the simpler interpretations and add: // SPEC QUESTION: [description].
- **Handling Gaps:** Check if a similar pattern is established. Choose the simpler of two reasonable approaches. Leave a comment explaining your choice.